



دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

داده‌ساختارها و الگوریتم‌ها

جلسه هفتم – تحلیل مرتب‌سازی ادغامی و روش‌های حل رابطه‌های بازگشتی

دکتر شهرام گلزاری

پاییز ۱۴۰۵

در پایان این جلسه انتظار داریم که:

- اجرای MERGE-SORT را روی یک آرایه دنبال کنیم و رابطه‌ی بازگشتی آن را بنویسیم.
- با **درخت بازگشت** نشان دهیم زمان مرتب‌سازی ادغامی $\Theta(n \lg n)$ است.
- هفت روش حل رابطه‌ی بازگشتی را بشناسیم و بدانیم هر کدام کجا به کار می‌آید.
- با **قضیه‌ی اصلی** زمان دو نسخه‌ی تابع `what` را تحلیل کنیم.
- رابطه‌ی بازگشتی **برج هانوی** را با **تکرار با جای‌گذاری** حل کنیم.

MERGE-SORT(A, p, r)

```

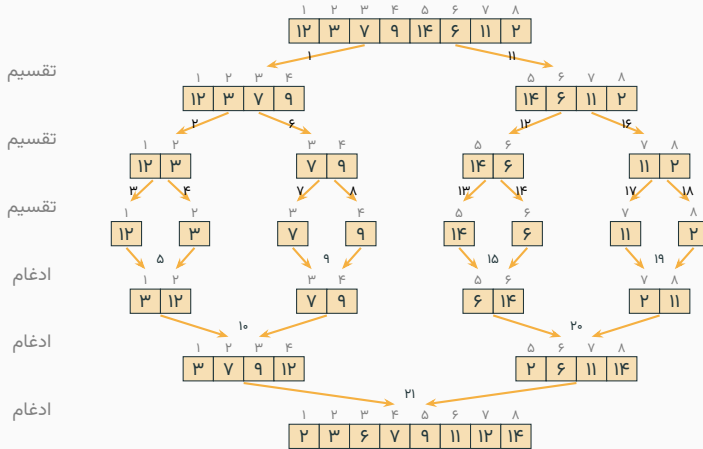
1  if  $p \geq r$                 // zero or one element?
2    return
3   $q = \lfloor (p + r) / 2 \rfloor$     // midpoint of  $A[p : r]$ 
4  MERGE-SORT( $A, p, q$ )        // recursively sort  $A[p : q]$ 
5  MERGE-SORT( $A, q + 1, r$ )    // recursively sort  $A[q + 1 : r]$ 
6  // Merge  $A[p : q]$  and  $A[q + 1 : r]$  into  $A[p : r]$ .
7  MERGE( $A, p, q, r$ )
    
```

۳۹ صفحه، CLRS, 4th ed.

در جلسه‌ی قبل:

- حالت پایه‌ی مرتب‌سازی ادغامی را شناختیم: اگر $p \geq r$ باشد، زیرآرایه حداکثر یک عنصر دارد و خودبه‌خود مرتب است (خط‌های ۱ و ۲).
- با رویه‌ی MERGE آشنا شدیم: دو زیرآرایه‌ی مجاور مرتب را در هم ادغام می‌کند (خط ۷).
- فهمیدیم هزینه‌ی زمانی MERGE روی n عنصر $\Theta(n)$ است.

رویهی MERGE-SORT



(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*

رابطه‌ی بازگشتی

«وقتی الگوریتمی یک فراخوانی بازگشتی دارد، اغلب می‌توانید زمان اجرایش را با یک **معادله‌ی بازگشتی** یا **رابطه‌ی بازگشتی** توصیف کنید، که زمان اجرای کل روی مسئله‌ای به اندازه‌ی n را برحسب زمان اجرای همان الگوریتم روی ورودی‌های کوچک‌تر توصیف می‌کند. سپس می‌توانید از ابزارهای ریاضی برای حل رابطه‌ی بازگشتی و ارائه‌ی کران‌هایی برای کارایی الگوریتم استفاده کنید.»

CLRS, 4th ed.، صفحه ۳۹ (ترجمه ماشینی)

$$T(n) = \begin{cases} \Theta(1), & n < n_0 \\ D(n) + aT(n/b) + C(n), & \text{در غیر این صورت} \end{cases}$$

بر اساس *CLRS, 4th ed.*، صفحه ۴۰

خط در MERGE-SORT	گام
خط ۳	$D(n) = \Theta(1)$
خط‌های ۴ و ۵	$2T(n/2)$
خط ۷	$C(n) = \Theta(n)$

«در این جا می‌بینید که رابطه‌ی بازگشتی برای $T(n)$ ، زمان اجرای بدترین حالت مرتب‌سازی ادغامی روی n عدد، چگونه نوشته می‌شود.

- **تقسیم:** گام تقسیم فقط وسط زیرآرایه را حساب می‌کند که زمان ثابتی می‌گیرد. بنابراین $D(n) = \Theta(1)$.
- **حل:** حل بازگشتی دو زیرمسئله، هر کدام به اندازه‌ی $n/2$ ، مقدار $2T(n/2)$ به زمان اجرا اضافه می‌کند (...).
- **ترکیب:** چون رویه‌ی MERGE روی یک زیرآرایه‌ی n عنصری زمان $\Theta(n)$ می‌گیرد، داریم $C(n) = \Theta(n)$.

CLRS, 4th ed.، صفحه ۴۱ (ترجمه ماشینی)

«افزودن $\Theta(n)$ به جمله‌ی $۲T(n/۲)$ از گام حل، رابطه‌ی بازگشتی زمان اجرای بدترین حالت $T(n)$ مرتب‌سازی ادغامی را به دست می‌دهد:»

$$T(n) = ۲T(n/۲) + \Theta(n) \quad (۲,۳)$$

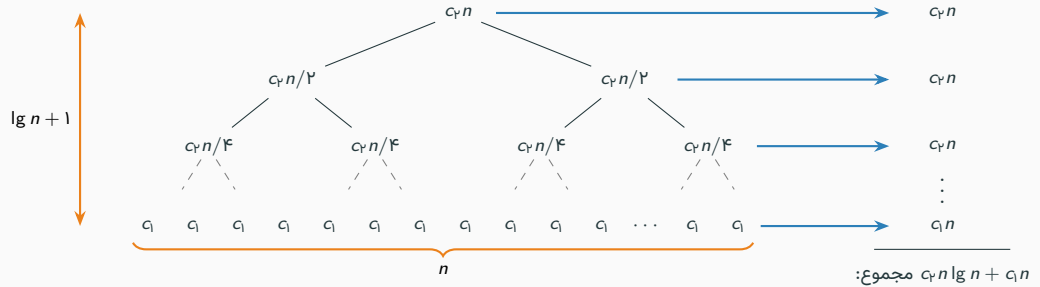
CLRS, 4th ed.، صفحه ۴۱ (ترجمه ماشینی)

درخت بازگشت

«برای سادگی، فرض کنید n توانی دقیق از ۲ است و حالت پایه‌ی ضمنی $n = 1$ است. آنگاه رابطه‌ی بازگشتی (۳.۲) در اصل چنین است:»

$$T(n) = \begin{cases} c_1, & n = 1 \\ 2T(n/2) + c_2n, & n > 1 \end{cases} \quad (2.4)$$

CLRS, 4th ed., صفحه ۴۲ (ترجمه ماشینی).



d) بخش ۵.۲، شکل ۴۳ (بازترسیم شکل ۵.۲، بخش ۵.۲)، *CLRS*, 4th ed.

«برای محاسبه‌ی هزینه‌ی کلی که رابطه‌ی بازگشتی (۴.۲) نشان می‌دهد، کافی است هزینه‌های همه‌ی سطرها را جمع کنید. درخت بازگشت $\lg n + 1$ سطح دارد. هزینه‌ی هر یک از سطرها بالای برگ‌ها $c_2 n$ است و هزینه‌ی سطح برگ‌ها $c_1 n$ ، که هزینه‌ی کل آن $c_2 n \lg n + c_1 n = \Theta(n \lg n)$ است.»

CLRS, 4th ed.، صفحه ۴۴ (ترجمه ماشینی)

$$T(n) = \underbrace{c_2 n + c_2 n + \dots + c_2 n}_{\lg n} + \underbrace{c_1 n}_{\text{برگ‌ها}} = c_2 n \lg n + c_1 n = \Theta(n \lg n)$$

روش‌های حل رابطه‌ی بازگشتی

روش	ایده
۱ تکرار با جای‌گذاری	باز کردن فرمول و به دست آوردن جواب به طور صریح.
۲ درخت بازگشت	بازکردن رابطه به شکل درخت: هزینه‌ی هر سطح درخت را حساب کن و همه‌ی سطرها را جمع بزن.
۳ حدس و استقرا	حدس خوب و استفاده از استقرا برای اثبات آن
۴ قضیه‌ی اصلی	برای $T(n) = aT(n/b) + f(n)$ با مقایسه‌ی $n^{\log_b a}$ و $f(n)$ جواب مستقیم به دست می‌آید.
۵ آکرا-بازی	تعمیم قضیه‌ی اصلی برای زیرمسئله‌هایی با اندازه‌های نابرابر، مانند $T(n) = T(n/3) + T(2n/3) + n$.
۶ معادله‌ی مشخصه (همگن و ناهمگن)	ریشه‌های معادله‌ی مشخصه شکل جواب را می‌دهند، مانند $T(n) = T(n-1) + T(n-2)$ ؛ اگر جمله‌ی اضافه داشته باشد، ناهمگن است.
۷ تابع مولد	دنباله‌ی جواب‌ها را ضریب‌های یک سری توانی بگیر، رابطه را به معادله‌ای برای این سری تبدیل کن و از حل آن جمله‌ی عمومی را به دست آور.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۹۵ (با تغییر و افزودن)

مثال: تابع what

هر دو تابع چه چیزی را حساب می‌کنند؟ هر دو a^n را حساب می‌کنند.

```
WHAT2( $a, n$ )  
1 if  $n == 0$   
2   return 1  
3 if  $n == 1$   
4   return  $a$   
5  $c = \text{WHAT2}(a, \lfloor n/2 \rfloor)$   
6 if  $n$  is even  
7   return  $c \cdot c$   
8 else return  $a \cdot c \cdot c$ 
```

```
WHAT1( $a, n$ )  
1 if  $n == 0$   
2   return 1  
3 if  $n == 1$   
4   return  $a$   
5  $c = \text{WHAT1}(a, \lfloor n/2 \rfloor)$   
6  $d = \text{WHAT1}(a, \lfloor n/2 \rfloor)$   
7 if  $n$  is even  
8   return  $c \cdot d$   
9 else return  $a \cdot c \cdot d$ 
```

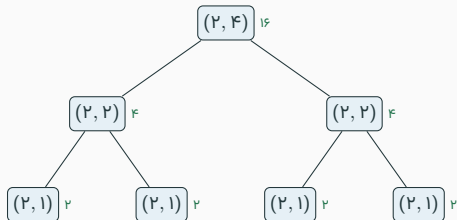
WHAT1 دو بار همان زیرمسئله را حل می‌کند ($c = d$)؛ WHAT2 یک بار حل می‌کند و جوابش را دو بار به کار می‌برد.

WHAT2



۳ فراخوانی

WHAT1



۷ فراخوانی

برای رابطه‌ی $T(n) = aT(n/b) + f(n)$ با ثابت‌های $a > 0$ و $b > 1$ ، تابع $f(n)$ را با $n^{\log_b a}$ مقایسه می‌کنیم:

حالت	شرط	جواب
۱	$f(n) = O(n^{\log_b a - \epsilon})$ برای یک $\epsilon > 0$	$T(n) = \Theta(n^{\log_b a})$
۲	$f(n) = \Theta(n^{\log_b a} \lg^k n)$ برای یک $k \geq 0$	$T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$
۳	$f(n) = \Omega(n^{\log_b a + \epsilon})$ برای یک $\epsilon > 0$ و $af(n/b) \leq cf(n)$ برای یک $c < 1$	$T(n) = \Theta(f(n))$

یادآوری: مرتب‌سازی ادغامی: $a = b = 2$ ، $n^{\log_2 2} = n$ و $f(n) = \Theta(n)$ حالت ۲ با $k = 0$ $\Theta(n \lg n)$.

بر اساس *CLRS, 4th ed.*، بخش ۴.۵، قضیه‌ی ۴.۱

WHAT2

```

WHAT2( $a, n$ )
1  if  $n == 0$ 
2    return 1
3  if  $n == 1$ 
4    return  $a$ 
5   $c = \text{WHAT2}(a, \lfloor n/2 \rfloor)$ 
6  if  $n$  is even
7    return  $c \cdot c$ 
8  else return  $a \cdot c \cdot c$ 
    
```

- خط‌های ۱ تا ۴ و ۶ تا ۸: زمان ثابت، یعنی $\Theta(1)$.
- خط ۵: فقط یک فراخوانی روی $\lfloor n/2 \rfloor$ ، یعنی $T(n/2)$.

$$T(n) = T(n/2) + \Theta(1)$$

کدام یک از حالت‌های قضیه اصلی است؟

$$T(n) = \Theta(\lg n)$$

WHAT1

```

WHAT1( $a, n$ )
1  if  $n == 0$ 
2    return 1
3  if  $n == 1$ 
4    return  $a$ 
5   $c = \text{WHAT1}(a, \lfloor n/2 \rfloor)$ 
6   $d = \text{WHAT1}(a, \lfloor n/2 \rfloor)$ 
7  if  $n$  is even
8    return  $c \cdot d$ 
9  else return  $a \cdot c \cdot d$ 
    
```

- خط‌های ۱ تا ۴ و ۷ تا ۹: زمان ثابت، یعنی $\Theta(1)$.
- خط‌های ۵ و ۶: دو فراخوانی روی $\lfloor n/2 \rfloor$ ، یعنی $2T(n/2)$.

$$T(n) = 2T(n/2) + \Theta(1)$$

کدام یک از حالت‌های قضیه اصلی است؟

$$T(n) = \Theta(n)$$

WHAT2

```

WHAT2( $a, n$ )
1  if  $n == 0$ 
2    return 1
3  if  $n == 1$ 
4    return  $a$ 
5   $c = \text{WHAT2}(a, \lfloor n/2 \rfloor)$ 
6  if  $n$  is even
7    return  $c \cdot c$ 
8  else return  $a \cdot c \cdot c$ 
    
```

- خط‌های ۱ تا ۴ و ۶ تا ۸: زمان ثابت، یعنی $\Theta(1)$.
- خط ۵: فقط یک فراخوانی روی $\lfloor n/2 \rfloor$ ، یعنی $T(n/2)$.

$$T(n) = T(n/2) + \Theta(1)$$

کدام یک از حالت‌های قضیه اصلی است؟

$$T(n) = \Theta(\lg n)$$

WHAT1

```

WHAT1( $a, n$ )
1  if  $n == 0$ 
2    return 1
3  if  $n == 1$ 
4    return  $a$ 
5   $c = \text{WHAT1}(a, \lfloor n/2 \rfloor)$ 
6   $d = \text{WHAT1}(a, \lfloor n/2 \rfloor)$ 
7  if  $n$  is even
8    return  $c \cdot d$ 
9  else return  $a \cdot c \cdot d$ 
    
```

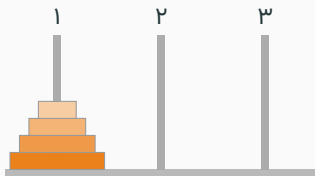
- خط‌های ۱ تا ۴ و ۷ تا ۹: زمان ثابت، یعنی $\Theta(1)$.
- خط‌های ۵ و ۶: دو فراخوانی روی $\lfloor n/2 \rfloor$ ، یعنی $2T(n/2)$.

$$T(n) = 2T(n/2) + \Theta(1)$$

کدام یک از حالت‌های قضیه اصلی است؟

$$T(n) = \Theta(n)$$

الگوریتم‌های بازگشتی: برج‌های هانوی



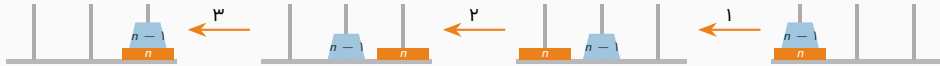
مثالی از مسئله‌ی برج‌های هانوی

«در این مسئله‌ی قدیمی و آشنا، n سکه‌ی سوراخ‌دار با قطرهای متفاوت به‌ترتیب قطرشان در میله‌ی شماره‌ی ۱ از ۳ میله‌ی موجود قرار دارند (سکه‌ی بزرگ‌تر در پایین سکه‌هاست). شکل ۳-۷ مثالی از حالت اولیه‌ی این مسئله را با ۴ سکه نشان می‌دهد. هدف، انتقال همه‌ی سکه‌ها به میله‌ی شماره‌ی ۳ به‌کمک میله‌ی شماره‌ی ۲ است به‌طوری‌که،

- هر بار فقط یک سکه جابه‌جا شود،
- در هر حرکت سکه‌ی کوچک‌تر بر روی سکه‌ی بزرگ‌تر قرار گیرد، و
- تعداد حرکت‌ها کمینه باشد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۸۴ و ۸۵ (بازترسیم شکل ۳-۷)

برای جابه‌جا کردن بزرگ‌ترین سکه، باید همه‌ی $n - 1$ سکه‌ی دیگر روی میله‌ی ۲ باشند. پس:



۱. $n - 1$ سکه‌ی بالایی را از میله‌ی ۱ به میله‌ی ۲ ببر .
۲. بزرگ‌ترین سکه را از میله‌ی ۱ به میله‌ی ۳ ببر .
۳. $n - 1$ سکه را از میله‌ی ۲ به میله‌ی ۳ ببر .

HA NOI(n, src, dst, tmp)

```
1 if  $n > 0$  then
2   HA NOI( $n - 1, src, tmp, dst$ )
3   move disk  $n$  from  $src$  to  $dst$ 
4   HA NOI( $n - 1, tmp, dst, src$ )
```

Erickson, Algorithms, صفحه ۲۶ (شکل ۱.۴)

اگر $T(n)$ تعداد حرکت‌ها باشد:

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n-1) + 1 & n > 1 \end{cases}$$

زیرمسئله‌ها $n - 1$ هستند، نه n/b ؛ پس قضیه‌ی اصلی این‌جا به کار نمی‌آید. از روش ۱، تکرار با جای‌گذاری، استفاده می‌کنیم.

$$\begin{aligned}T(n) &= 2T(n-1) + 1 \\&= 2[2T(n-2) + 1] + 1 = 2^2 T(n-2) + 2 + 1 \\&= 2^2 [2T(n-3) + 1] + 2 + 1 = 2^3 T(n-3) + 2^2 + 2 + 1 \\&\vdots \\&= 2^k T(n-k) + (2^{k-1} + \dots + 2 + 1) = 2^k T(n-k) + (2^k - 1)\end{aligned}$$

به حالت پایه می‌رسیم وقتی $n-k=1$ ، یعنی $k=n-1$:

$$T(n) = 2^{n-1} \cdot 1 + 2^{n-1} - 1 = 2^n - 1 = \Theta(2^n)$$

منابع

- Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms* (CLRS), 4th Edition, MIT Press, 2022
- محمد قدسی — داده‌ساختارها و مبانی الگوریتم‌ها، انتشارات فاطمی، چاپ ۱۳۹۷
- Jeff Erickson — *Algorithms*, 2019